

从串行到并行：AI Agent工作流的高效协同设计

第三章 Parallelization

目录

CONTENT



01 并行化原理与核心机制
Principles and Mechanisms

02 应用场景与案例分析
Applications and Case Studies

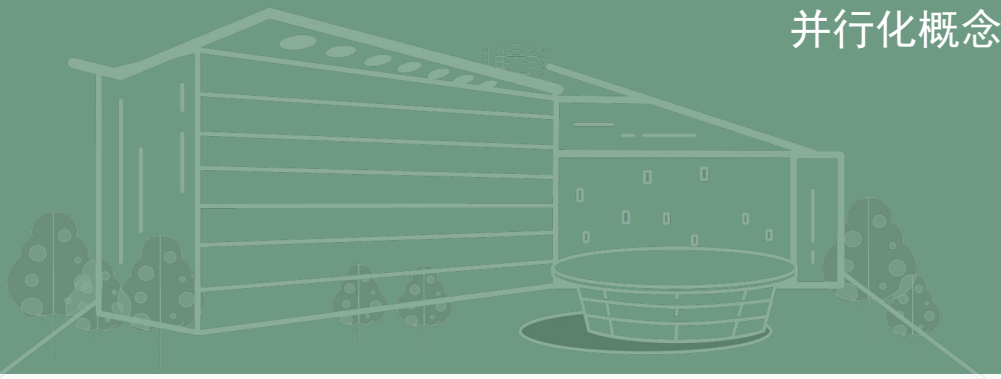
03 框架实现代码分析与复现
Implementation and Practice

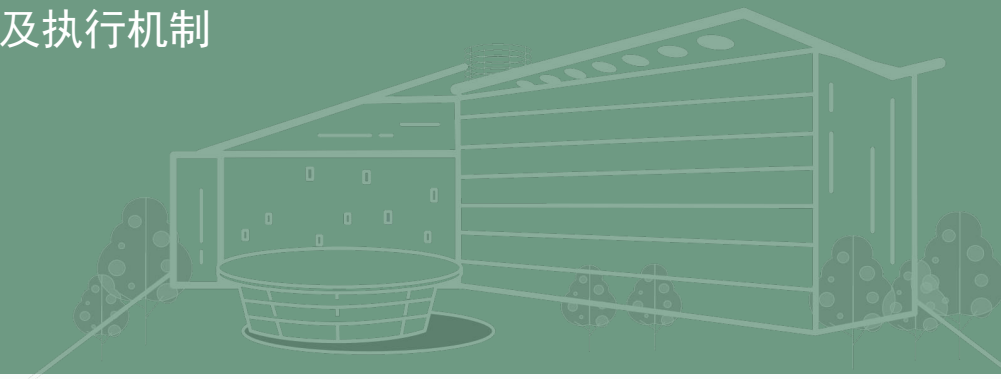
04 综合对比与总结展望
Comparison and Conclusions

01

并行化原理与核心机制

并行化概念、串行与并行对比、任务拆分及执行机制





華中師範大學

为什么需要并行化？

研究背景：从“顺序执行”、“依次等待”走向“独立任务同时推进”

复杂Agent任务经常包含多个处理环节。若相互独立的操作仍逐项串行执行，等待时间便会不断累积，降低系统响应效率。

01 提示链 Prompt Chaining

顺序连接处理步骤

A → B → C

02 路由 Routing

根据条件选择不同路径

条件 → 分支选择

03 并行化 Parallelization

让独立子任务同时推进

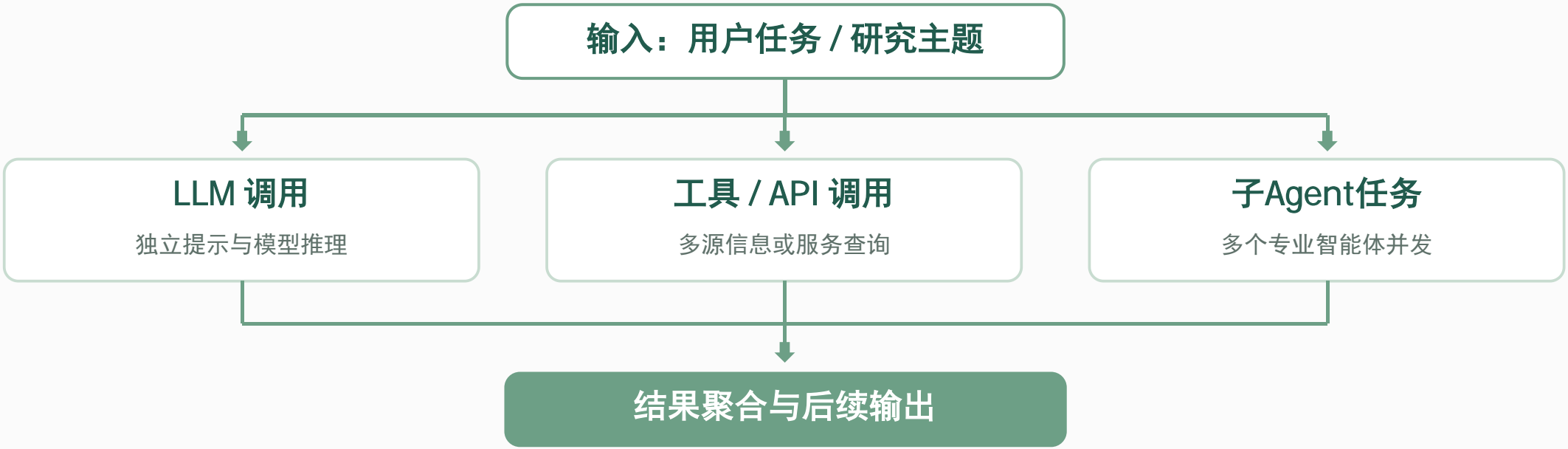
A // B // C

解决问题：不必等待的任务就不要依次等待，以并发执行减少整体延迟。

Parallelization：并行化模式的定义

并发执行独立的LLM调用、工具调用，或整个子Agent任务

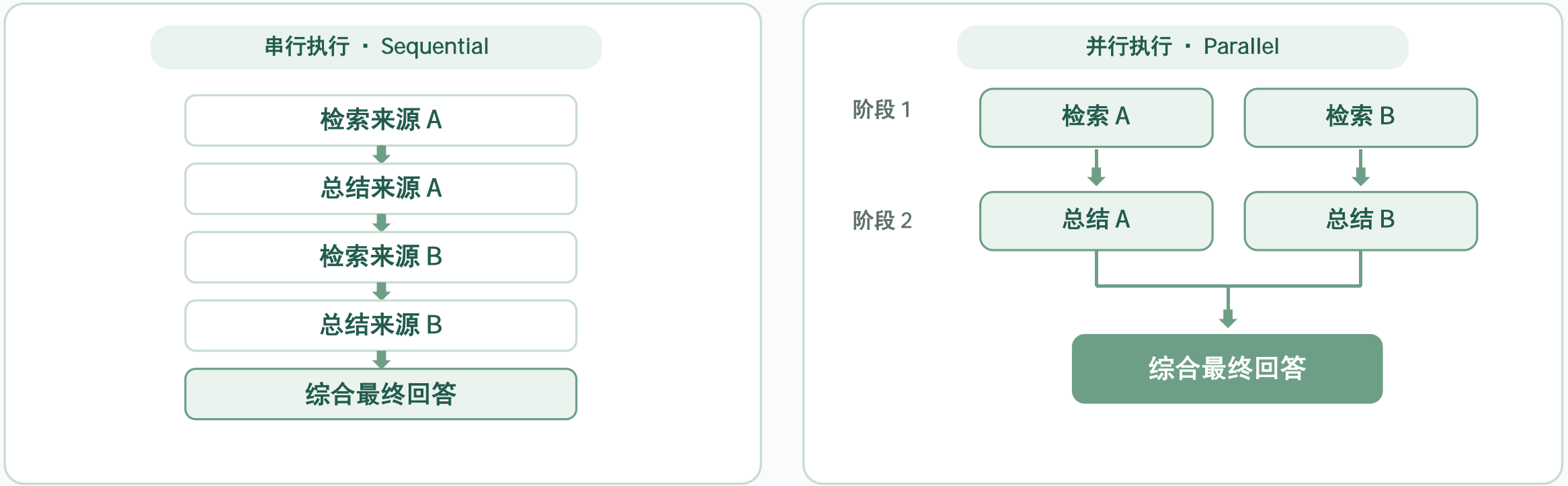
并行化的核心：识别工作流中不依赖其他步骤即时输出的环节，使其并发执行，并在需要时汇总结果。



图中展示可并发的组件类型，并不意味着所有场景都需要同时采用这三类组件。

串行执行 vs 并行执行

教材原例：检索来源A与来源B，分别总结，再综合最终回答



串行时间 ≈ 所有步骤耗时之和

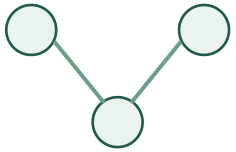
并行阶段 ≈ 最慢分支耗时 + 额外开销

并行化的核心运行机制

三个连续环节：独立任务识别 → 并发执行 → 结果聚合

识别独立任务

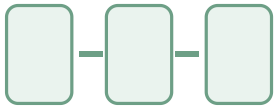
Task Independence



- 分析子任务的输入输出关系
- 识别没有直接数据依赖的环节
- 确定可同时启动的任务集合

并发执行子任务

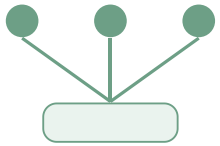
Concurrent Execution



- 使用异步机制或并行编排框架
- 同时推进多个LLM / API调用
- 必要时让多个专业Agent协作

汇总执行结果

Result Aggregation



- 收集各个并行分支的输出
- 等待后续步骤所需结果就绪
- 整合信息并进入下一处理阶段

关键：并行化不是无条件同时执行，而是根据依赖关系组合并行阶段与串行阶段。

并行化的适用条件与技术支撑

是否存在不依赖其他任务即时输出、能够同时启动的操作？

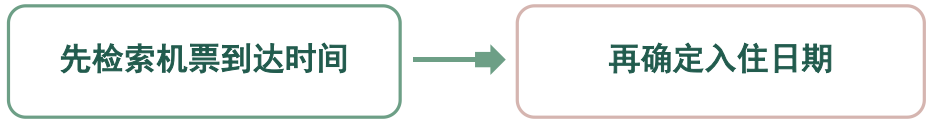
判断依据：任务B是否必须使用任务A尚未产生的输出？

可以并行：相互独立



多个独立API / 数据库查询，可以同时发起。

不能并行：存在依赖



后一步依赖前一步产出时，应按顺序处理。

教材提及的并行编排框架

LangChain / LCEL

RunnableParallel：组织多个Runnable

LangGraph

用图中的并行分支与汇合节点

Google ADK

通过ParallelAgent并发执行子Agent

典型价值：面对LLM推理、工具调用和外部服务等I/O等待任务，减少串行等待造成的延迟。

并行智能体的设计步骤

以旅行规划为例：从任务依赖到效果验证

设计步骤	关键问题	旅行规划示例
01 **明确输入**	哪些共同条件必须先确定？	目的地、日期、人数与预算
02 **分析依赖**	哪些任务可以独立执行？	条件确定后，并行查询机票、酒店与活动
03 **定义输出**	各分支返回什么，怎样对应？	统一返回价格、时间、来源与查询状态
04 **设置执行规则**	并发数量、超时和失败如何处理？	限制并发数；设置超时、重试与备选来源
05 **设计汇总**	如何处理冲突和缺失？	核对预算与时间衔接，标明缺失信息
06 **验证效果**	是否更快，质量是否保持？	比较耗时、成功率、费用与方案完整度

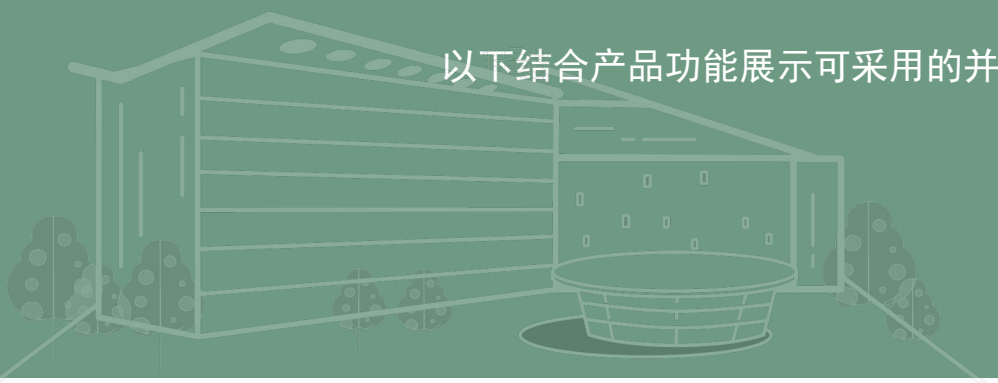
设计要点：先判断依赖，再组织并发，最后用实测检验收益。

02

应用场景与案例分析

教材案例与实际应用

以下结合产品功能展示可采用的并行化设计，流程图为本组分析示意，不代表产品公开的内部实现。





华中師範大學

用WindClaw智能体研究比亚迪公司



1. 信息收集与研究

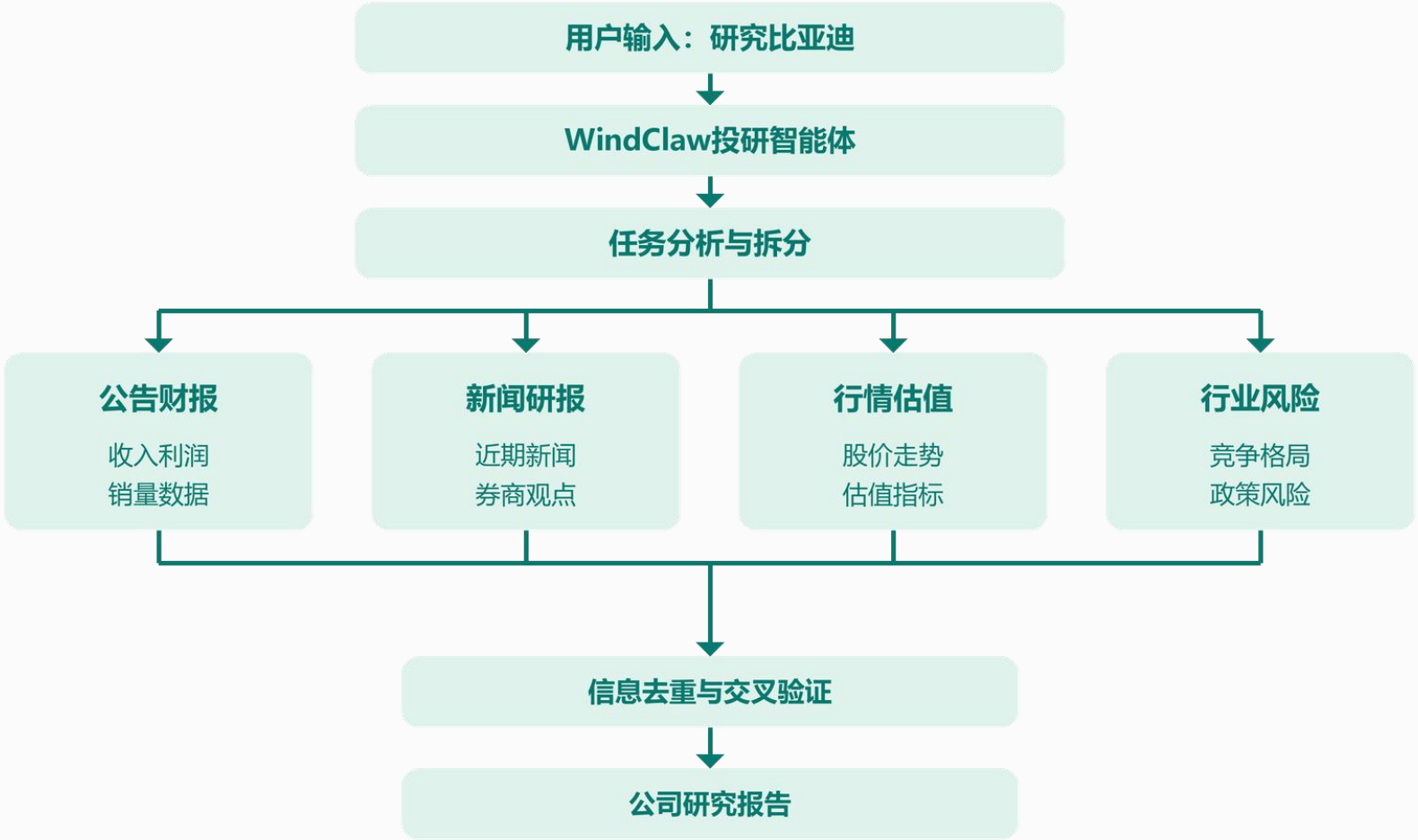
并行任务： 同时分析比亚迪的公告财报、检索近期新闻与券商研报、获取股价及估值数据，并评估行业竞争和政策风险。

优点： 多个研究任务同步执行，并通过信息去重与交叉验证，更快汇集多源研究材料。

02 实际应用与用例-1

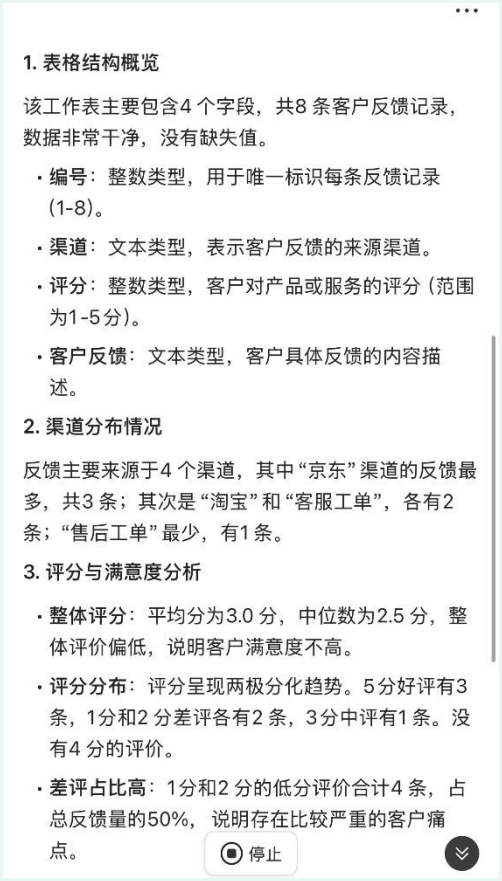


信息收集与研究流程图



WPS AI分析客户反馈





2. 数据处理与分析

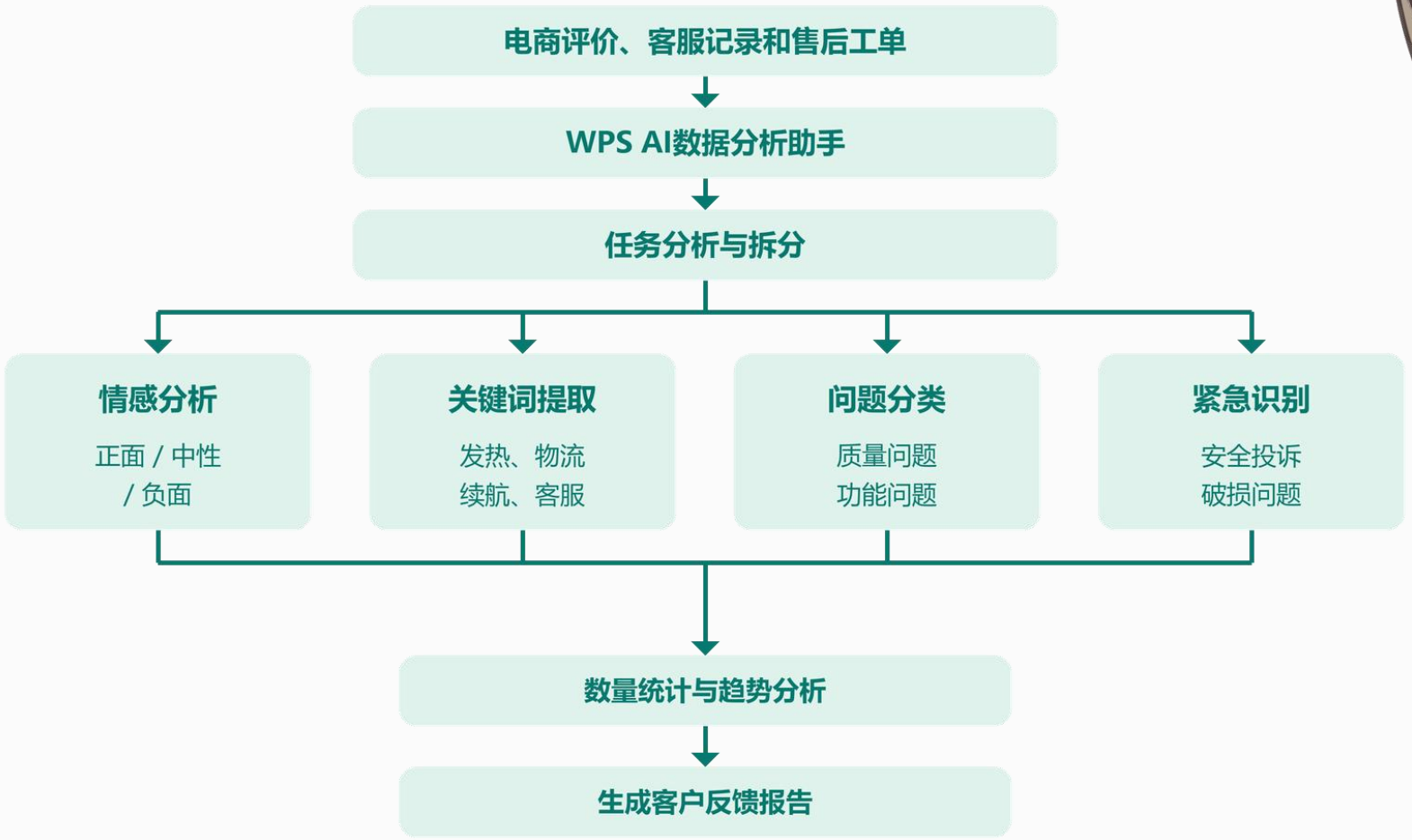
并行任务： 同时对客户反馈运行情感分析、关键词提取、问题分类和紧急问题识别，并统计不同渠道和问题类型的数据。

优点： 快速从大量客户反馈中获得多维度分析结果，及时发现集中投诉和需要优先处理的产品问题。

02 实际应用与用例-2



数据处理与分析流程图



实际应用与用例-3

携程智能体旅行规划





3. 多API或工具交互

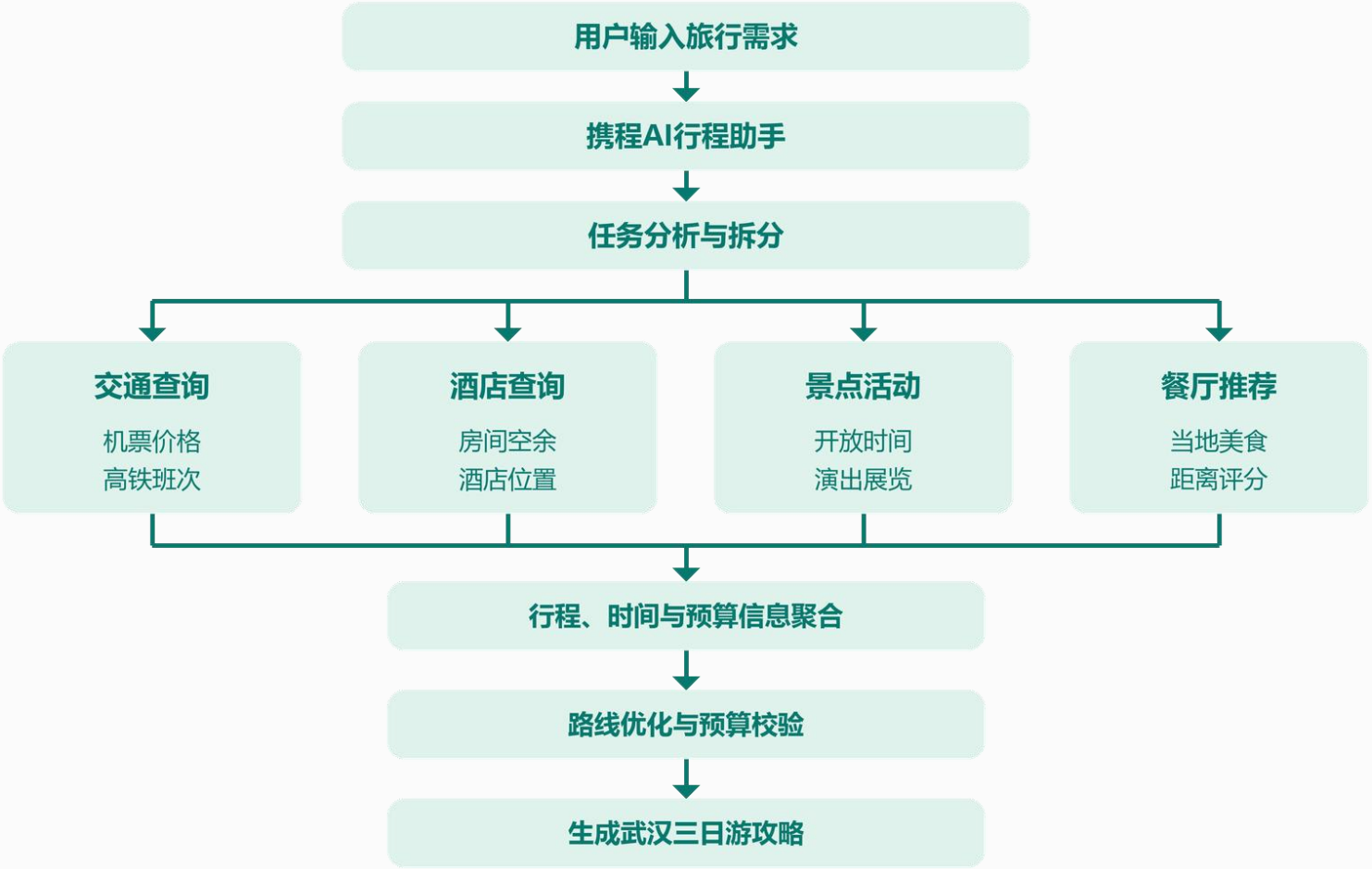
并行任务：同时查机票、搜酒店空房、查当地活动、找餐厅推荐

优点：更快呈现完整行程方案。

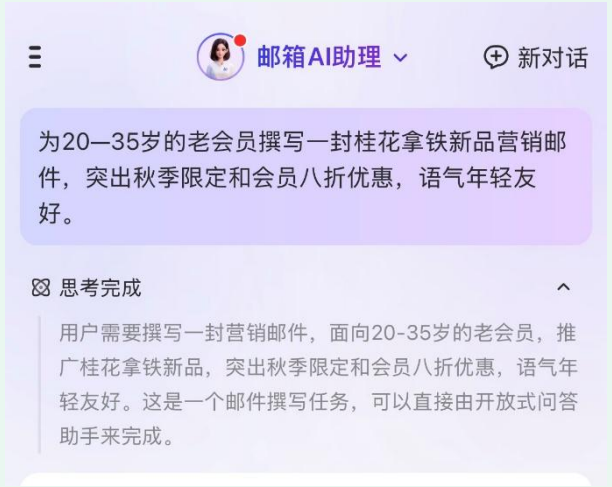
02 实际应用与用例-3

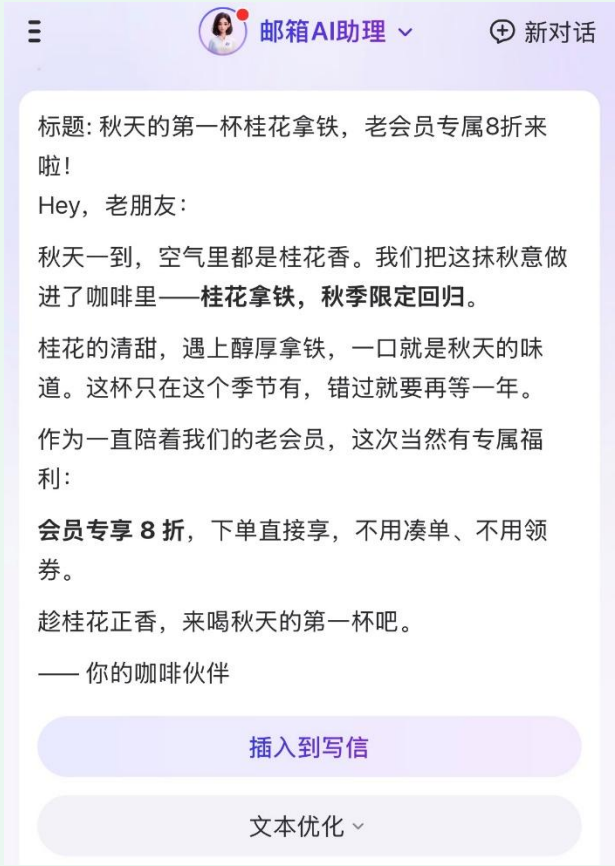


多API或工具交互流程图



网易邮箱大师智能体撰写邮件





4. 多组件内容生成

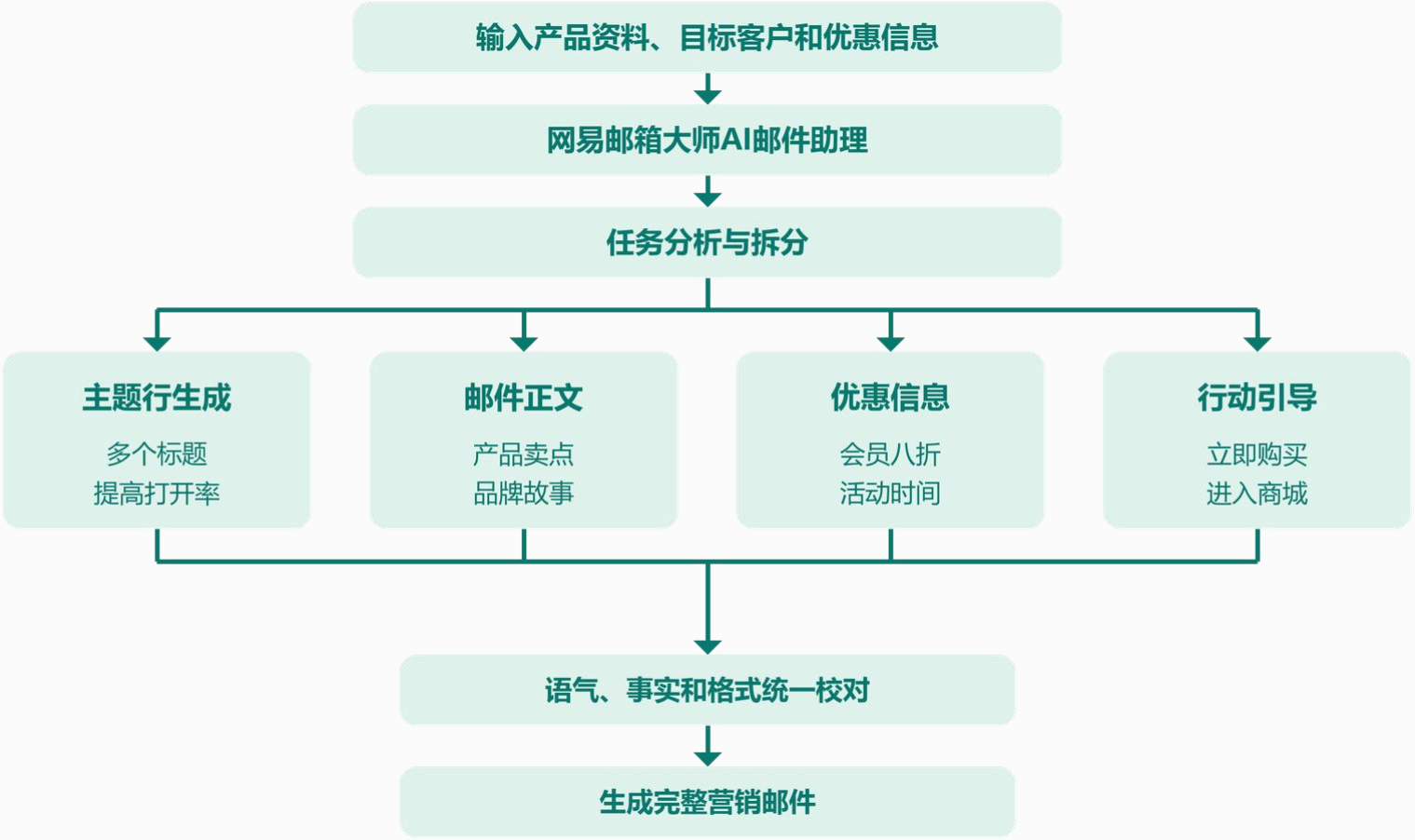
并行任务： 同时生成邮件主题行、营销正文、优惠信息和行动引导，并分别进行语气与内容优化。

优点： 并行生成各组成部分，再统一调整语气与内容，同时可以生成多个主题和正文版本供选择。

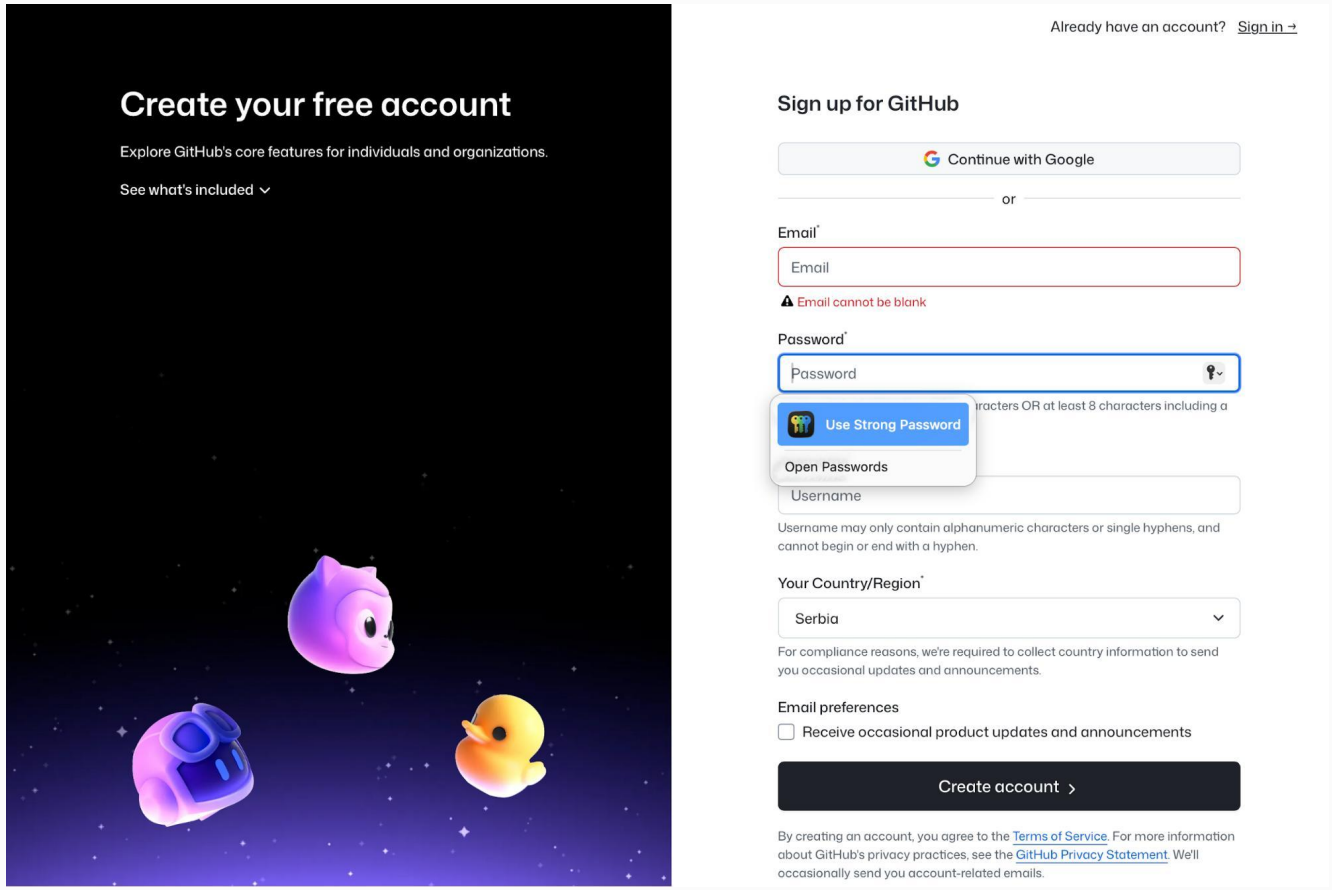
02 实际应用与用例-4



多组件内容生成流程图



GitHub注册信息校验

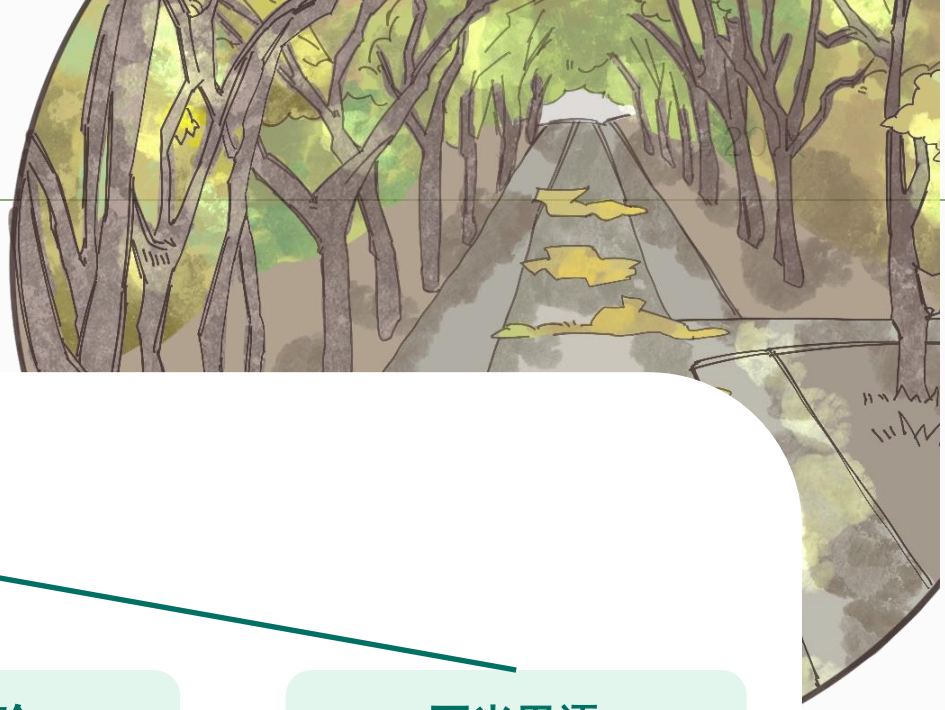


5.应用说明

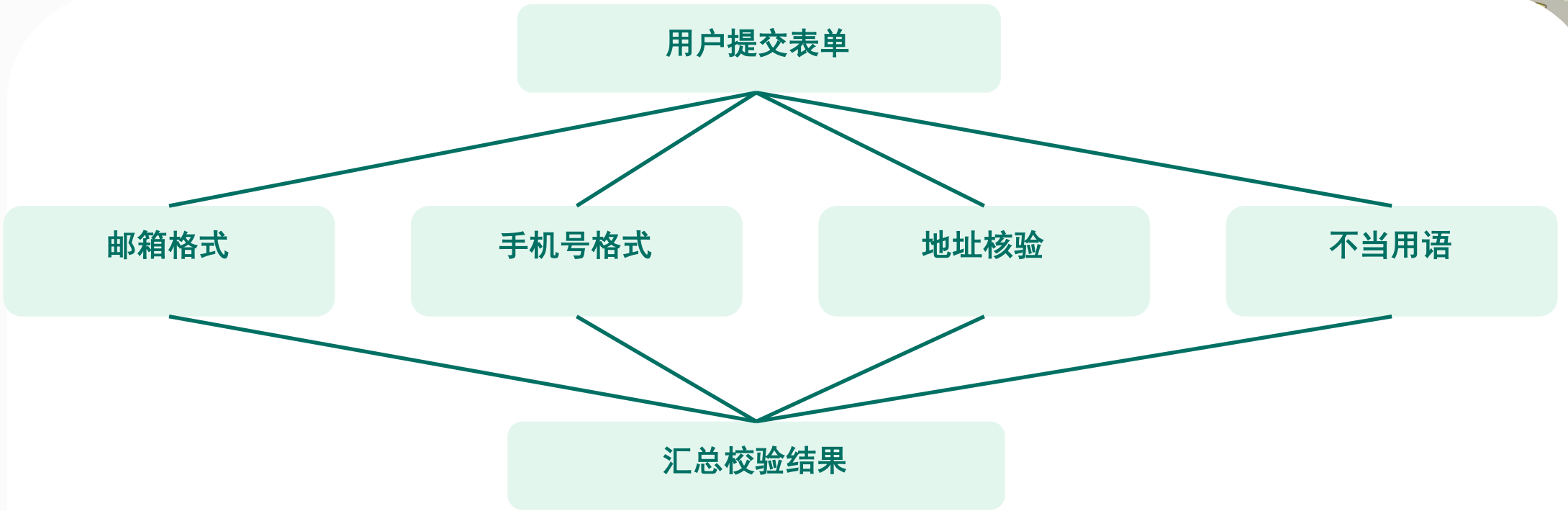
- 邮箱：非空及格式检查
- 密码：复杂度与长度规则
- 用户名：字符规范检查
- 汇总校验结果并反馈

02

实际应用与用例-5

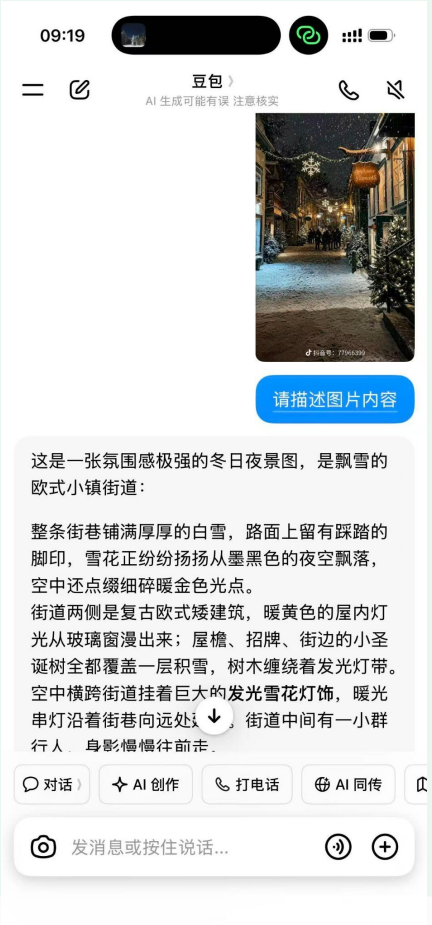


并行校验流程



全部通过 → 接收输入　任一失败 → 返回对应错误提示

豆包图文理解



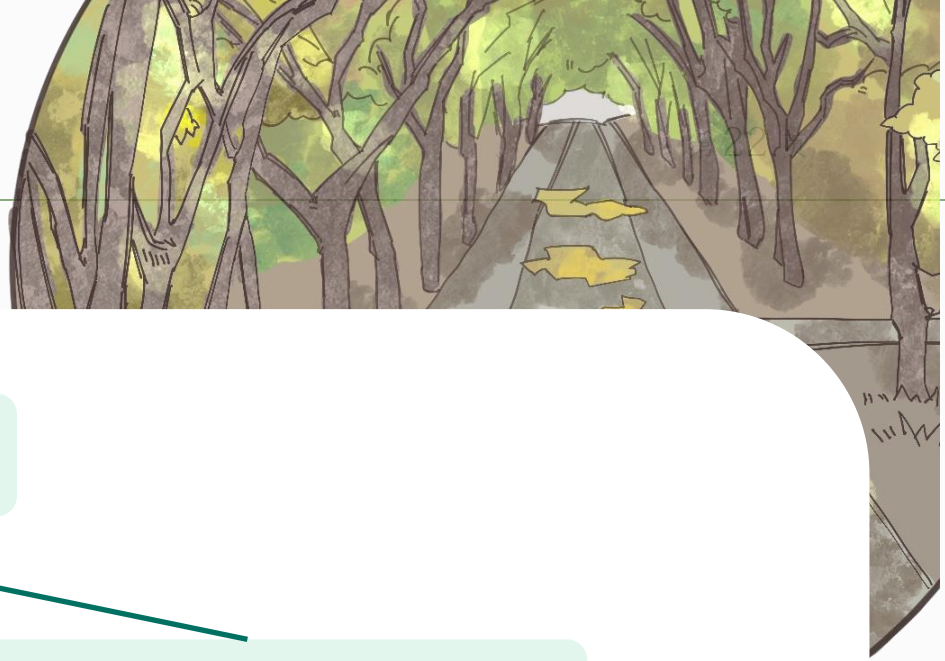
6.操作过程

- 上传冬日街景图片
- 输入文字：“请描述图片内容”
- 豆包结合图片与问题，输出场景描述

02

实际应用与用例-6

多模态处理流程



输入：文字 + 图片

文本分析分支

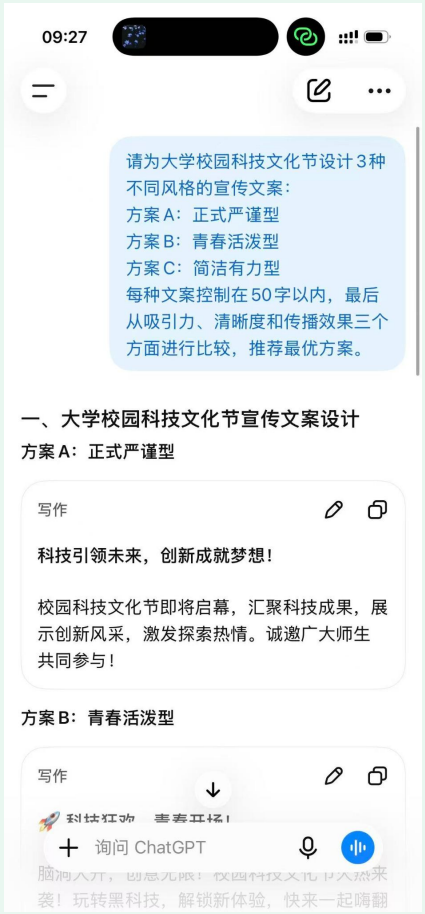
情感倾向识别
关键词提取

图像分析分支

物体识别
场景描述

融合文本与图像结果

ChatGPT多方案生成





7.案例分析

① 生成三种风格文案

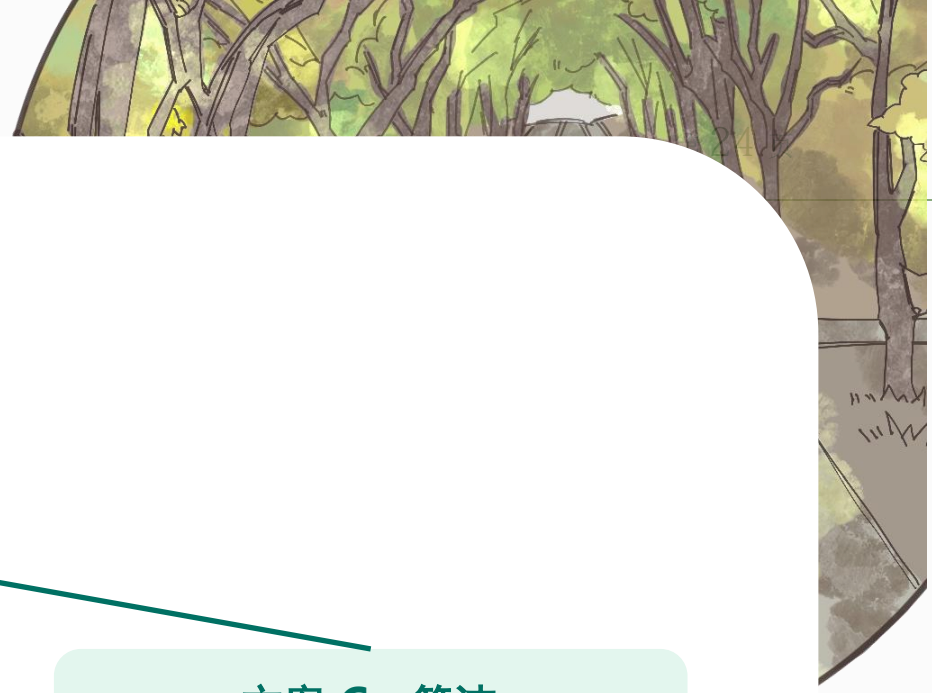
② 比较吸引力、清晰度和传播效果

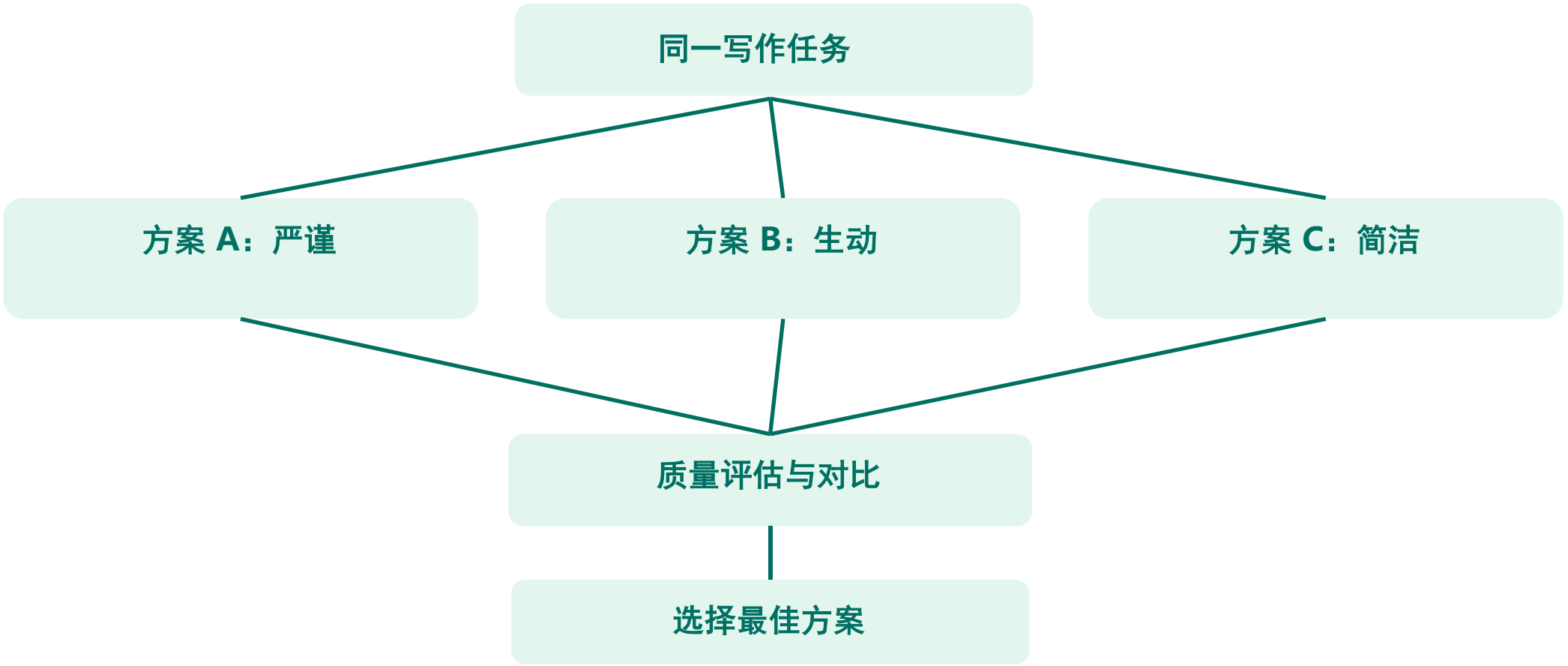
③ 推荐方案B：青春活泼型

并行化设计：不同文案生成任务可并行，再统一评价择优。

实际应用与用例-7

多方案生成与评估流程

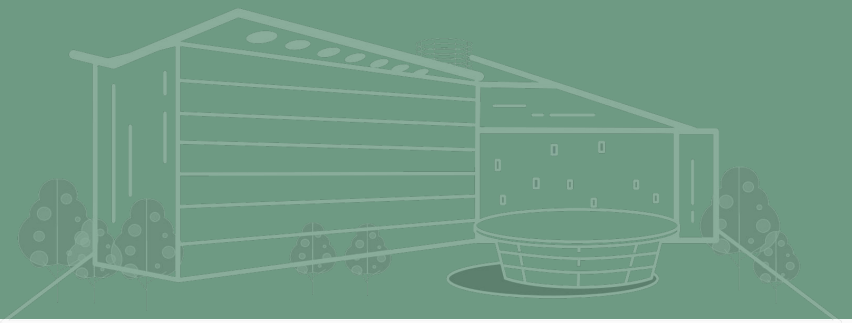


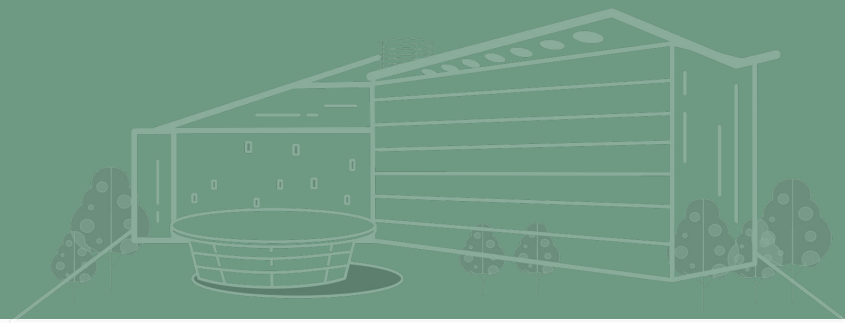


03

框架实现代码分析与复现

Google ADK与LangChain



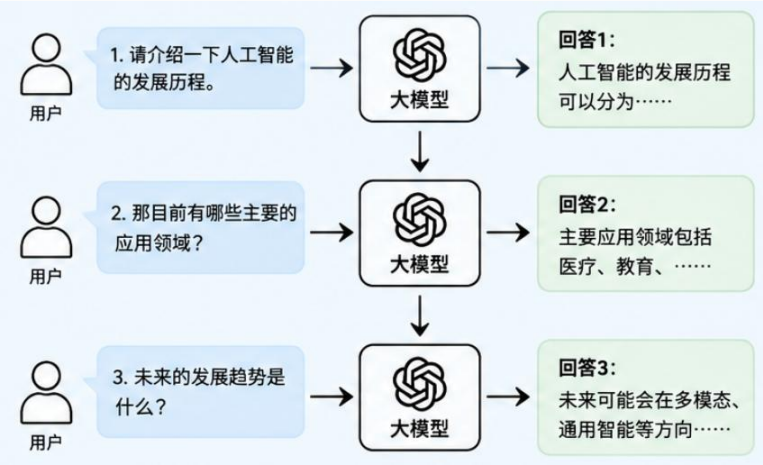


華中師範大學

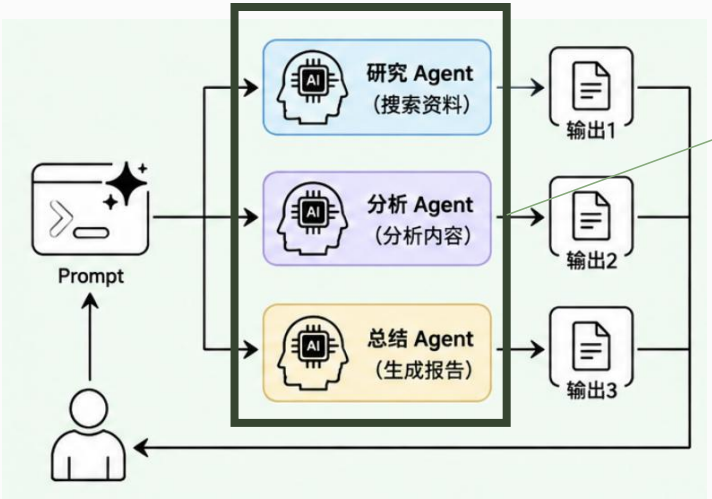
01 Google ADK (Agent Development Kit)

Google 推出的一个用于构建 AI Agent 的开发框架。

普通调用大模型流程（单线）



使用ADK后并行化的智能体



Parallelization

让多个彼此独立的任务同时执行，而不是等待前一个任务执行结束后再开始。

支持多智能体协作

多个智能体分工处理任务

支持流程编排

串行 / 并行 / 条件分支

支持工具调用

搜索 / 数据库 / API 等

ADK 更适合处理复杂任务，而不仅仅是一次简单问答。

01 Google ADK 里面的三个核心 Agent

LLMAgent

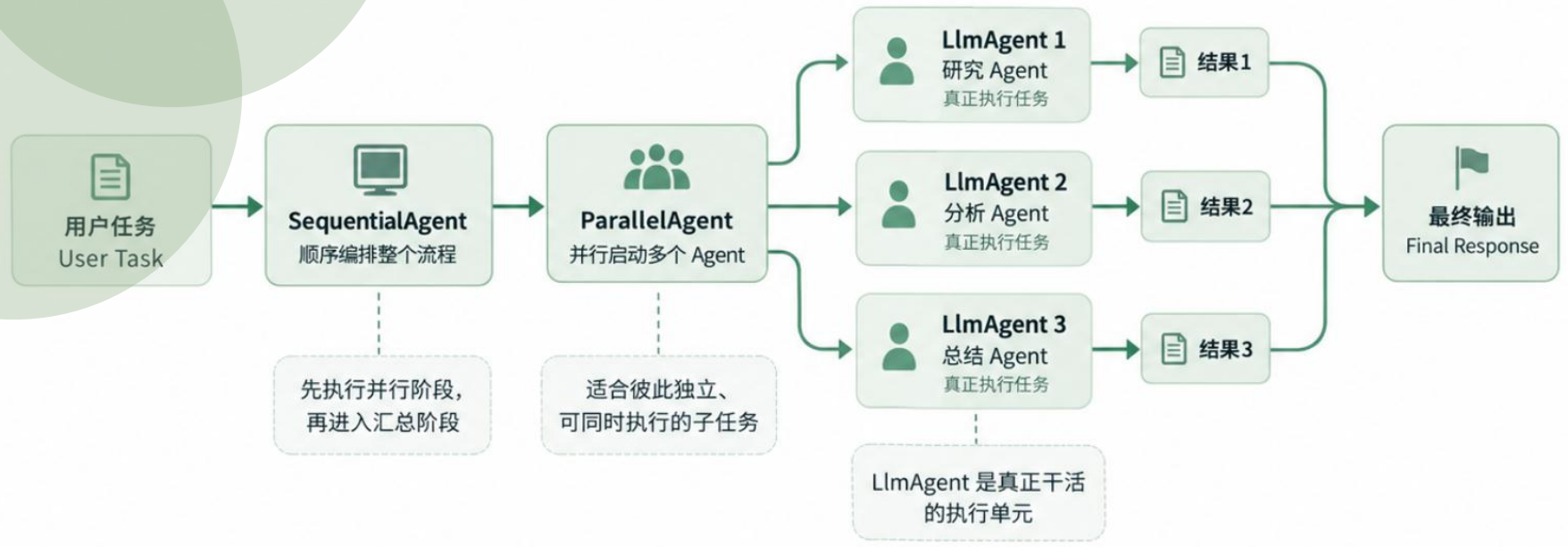
真正执行任务

SequentialAgent

按顺序执行 Agent

ParallelAgent

同时启动多个 Agent



01 代码分析：先设定了三个可并行的LlmAgent

定义三个独立研究 Agent，分别负责不同任务。

```python
# ---------- Agent 1：研究可再生能源 ----------
researcher_agent_1 = LlmAgent(
    name="RenewableEnergyResearcher",

    model=GEMINI_MODEL,

    instruction="""
You are an AI Research Assistant specializing in energy.

Research the latest advancements in renewable energy sources.

Use the Google Search tool provided.

Summarize your key findings concisely in 1-2 sentences.

Output only the summary.
""",

    description="Researches renewable energy sources.",

    tools=[google_search],

    # 将结果保存到 session state 中
    output_key="renewable_energy_result",
)
```

```python
# ---------- Agent 2：研究电动车技术 ----------
researcher_agent_2 = LlmAgent(
    name="EVResearcher",

    model=GEMINI_MODEL,

    instruction="""
You are an AI Research Assistant specializing in transportation.

Research the latest developments in electric vehicle technology.

Use the Google Search tool provided.

Summarize your key findings concisely in 1-2 sentences.

Output only the summary.
""",

    description="Researches electric vehicle technology.",

    tools=[google_search],

    # 将结果保存到 session state 中
    output_key="ev_technology_result",
)
```

```python
# ---------- Agent 3：研究碳捕集技术 ----------
researcher_agent_3 = LlmAgent(
    name="CarbonCaptureResearcher",

    model=GEMINI_MODEL,

    instruction="""
You are an AI Research Assistant specializing in climate solution

Research the current state of carbon capture methods.

Use the Google Search tool provided.

Summarize your key findings concisely in 1-2 sentences.

Output only the summary.
""",

    description="Researches carbon capture methods.",

    tools=[google_search],

    # 将结果保存到 session state 中
    output_key="carbon_capture_result",
)
```

ParallelAgent 同时调度多个互不依赖的子 Agent

```python
# ============================================================
# 3. 创建 ParallelAgent
#    三个研究 Agent 会在这里并行执行
# ============================================================

parallel_research_agent = ParallelAgent(
    name="ParallelWebResearchAgent",

    sub_agents=[
        researcher_agent_1,
        researcher_agent_2,
        researcher_agent_3,
    ],

    description="Runs multiple research agents in parallel to gather information.",
)
```

调度前面三个LlmAgent

先并行采集，再串行汇总。

```python
sequential_pipeline_agent = SequentialAgent(
    name="ResearchAndSynthesisPipeline",

    sub_agents=[
        parallel_research_agent,
        merger_agent,
    ],

    description=(
        "Coordinates parallel research "
        "and synthesizes the results."
    ),
)
```

```python
# ============================================================
# 4. 定义总结 Agent
#    它会读取前三个 Agent 保存的结果，并进行汇总
# ============================================================

merger_agent = LlmAgent(
    name="SynthesisAgent",

    model=GEMINI_MODEL,

    instruction="""
You are an AI Assistant responsible for combining research findings
into a structured report.

Your task is to synthesize the following research summaries.

IMPORTANT:
Your response MUST be based exclusively on the information provided
in the input summaries below.

Do NOT add external knowledge, facts, or details.

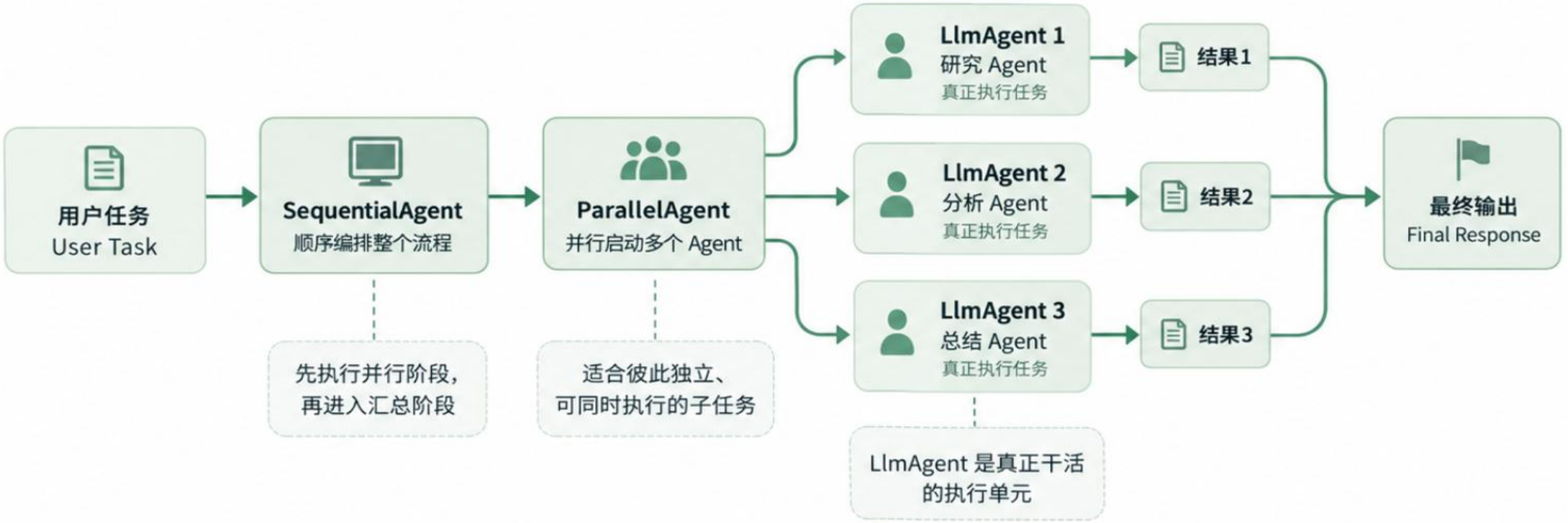
Input Summaries:

Renewable Energy:
{renewable_energy_result}

Electric Vehicles:
{ev_technology_result}
```

提示词

01 总结



1 SequentialAgent

顺序编排整个流程

- 负责整体任务的规划与流程控制
- 先执行并行阶段，再进入汇总阶段
- 按顺序组织各个步骤，管理任务流转

2 ParallelAgent

并行启动多个 Agent

- 将任务拆分为多个可并行执行的子任务
- 同时启动多个 LlmAgent
- 适合彼此独立、可同时执行的子任务
- 提高整体任务处理效率

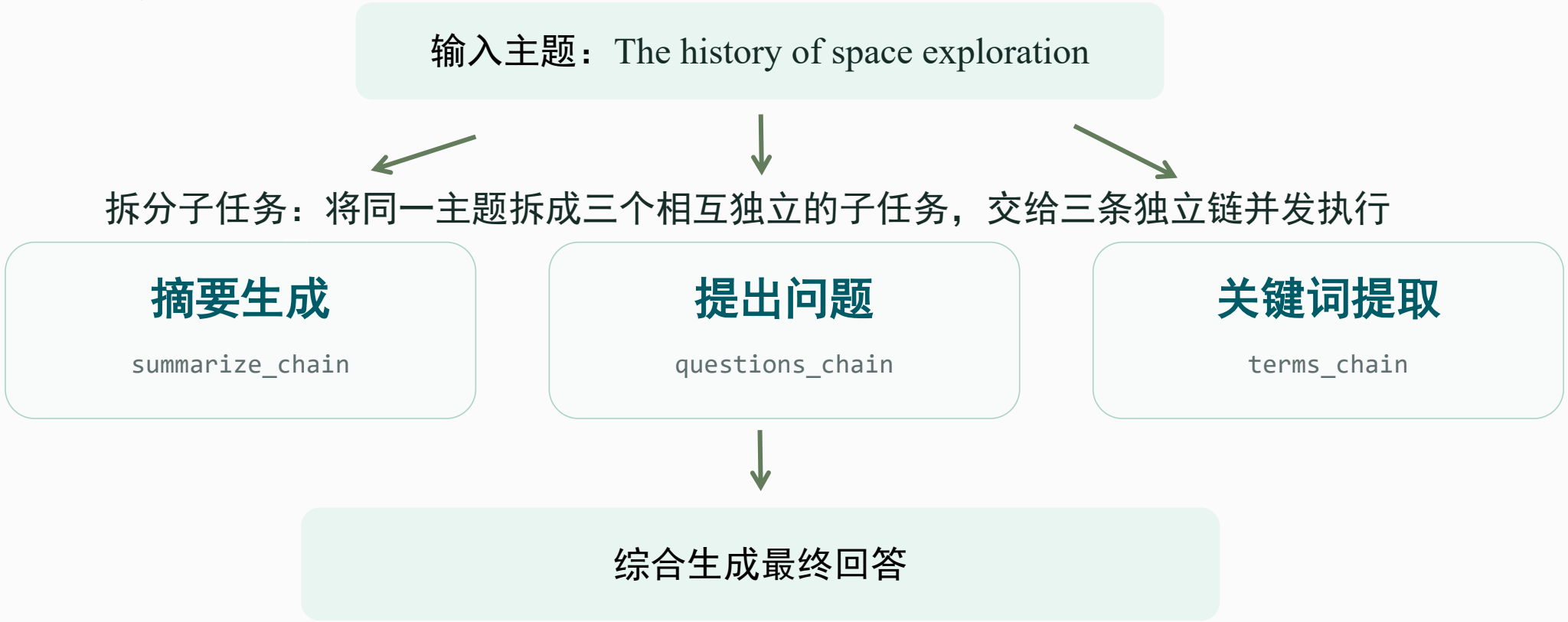
3 LlmAgent

真正执行任务的单元

- 具体执行各类任务的工作单元
- 可以是研究、分析、总结等不同角色
- 接收子任务并独立执行
- 输出各自的任务结果

课本案例介绍LangChain 并行化

案例背景：以 The history of space exploration（太空探索的历史）为主题，要求大模型从不同角度分析同一主题，并将结果整合成一份综合回答。



核心思路：并发+顺序执行，即三个子任务并发执行；综合回答需等待三个子任务完成。适用于子任务之间无依赖。

代码全貌：五个模块组成一条完整工作流

01	02	03	04	05
模型初始化	**构建三条任务链**	**并发调度**	**综合生成**	**执行与输出**
`ChatOpenAI`	`Prompt | LLM | Parser`	`RunnableParallel`	`synthesis_prompt | llm`	`ainvoke(topic)`

从代码模块到实际执行：三个阶段

01 准备阶段

初始化模型，设置各自的 Prompt，定义三条独立任务链。

这一步建立工作流结构，还没有开始处理输入主题。

02 并发阶段

RunnableParallel 将同一个 topic 传给三条链，并发得到摘要、问题与关键词。

三条任务彼此独立；最慢的任务决定并发阶段何时结束。

03 综合阶段

将结果字典填入 synthesis_prompt，再调用模型生成一份综合回答。

ainvoke() 异步启动整条链，等待最终结果返回。

代码拆解 1：三条任务链

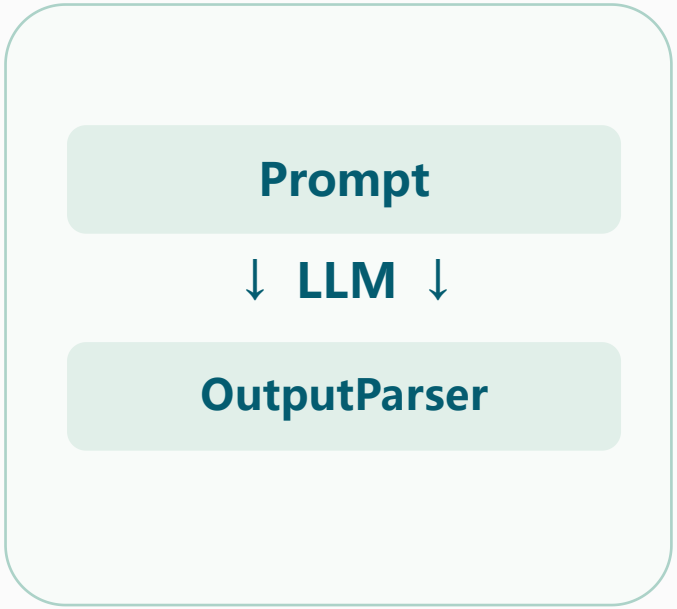
链名	System Prompt	输出
summarize_chain	Summarize the following topic concisely	简短摘要
questions_chain	Generate three interesting questions about the following topic	三个问题
terms_chain	Identify 5-10 key terms from the following topic, separated by commas	5-10 个关键词

02 代码拆解 1：定义三条任务链

三个任务只改变提示词，代码结构相同，这里以摘要链为例

```
# 摘要任务链
summarize_chain = (
ChatPromptTemplate.from_messages([
("system", "Summarize ... concisely:"),
("user", "{topic}"),
])
| llm
| StrOutputParser()
)
```

三条链的共同结构



Prompt（组织输入） → LLM（生成内容） → Parser（提取字符串）

代码拆解 2：RunnableParallel并发执行

关键：将独立任务交给并行执行器

```
# 并发调度器
map_chain = RunnableParallel({
"summary": summarize_chain,
"questions": questions_chain,
"key_terms": terms_chain,
"topic": RunnablePassthrough(),
})
```

并发调用三条链

保留原始输入主题

```
输出结构示例 (JSON):
{
"summary": "...",
"questions": "...",
"key_terms": "...",
"topic": "The history of space exploration"
}
```

要点：三条链并发执行，互不等待对方结果；
RunnablePassthrough() 把原始 topic 原样透传，留给下游综合用。

02 代码拆解 3：结果整合和异步调用

先汇总并行结果，再调用模型生成最终回答

```
# 1 综合提示词：吃进字典四项
synthesis_prompt = ChatPromptTemplate
.from_messages([...])
# 2 拼装完整链：并发 → 综合
full_parallel_chain = (
map_chain
| synthesis_prompt
| llm | StrOutputParser()
)
# 3 异步启动
response = await full_parallel_chain
.ainvoke(topic)
```

代码执行顺序（5 步）

① RunnableParallel 并发执行三条链，共 3 次 LLM 调用

② 汇总为字典：summary / questions / key_terms / topic

③ synthesis_prompt 接收字典并填充模板（含 4 个占位符）

④ 调用第 4 次 LLM 生成综合回答

⑤ StrOutputParser() 输出最终字符串

说明：response = await full_parallel_chain.ainvoke(topic) 表示异步执行整条链并等待结果；ainvoke 需放在 async 函数里，脚本入口通常用 asyncio.run(...)。

01 串并行对比代码

```python
# 串行执行
async def run_serial(topic):
    summary = await summary_chain.ainvoke(topic)
    questions = await questions_chain.ainvoke(topic)
    key_terms = await terms_chain.ainvoke(topic)

    results = {
        "summary": summary,
        "questions": questions,
        "key_terms": key_terms,
        "topic": topic
    }

    return await synthesis_chain.ainvoke(results)


# 并行执行
parallel_chain = RunnableParallel({
    "summary": summary_chain,
    "questions": questions_chain,
    "key_terms": terms_chain,
    "topic": RunnablePassthrough()
})
```

02 实验设置

测试主题： 太空探索的历史

模型： qwen-plus

实验任务： 摘要生成、问题生成、关键词提取

对比方式： 串行执行 / RunnableParallel

03 结果呈现

```
========== 串行子任务 ==========
摘要生成：6.07 秒
问题生成：4.01 秒
关键词提取：13.38 秒

========== 并行子任务 ==========
三个子任务并行耗时：13.55 秒

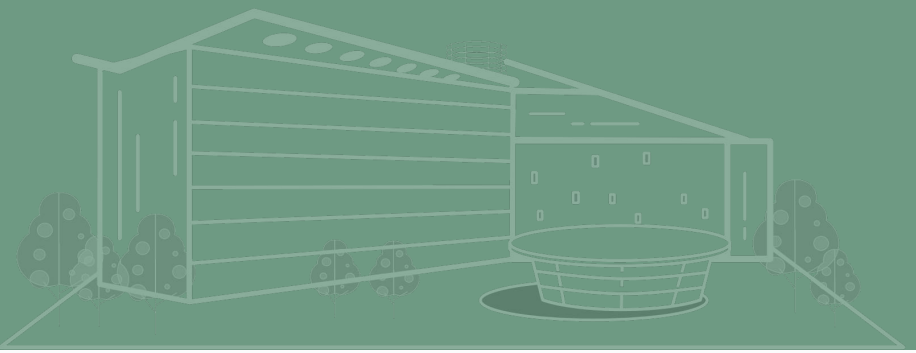
========== 对比结果 ==========
串行子任务总耗时：23.46 秒
并行子任务总耗时：13.55 秒
子任务阶段加速比：1.73 倍
```

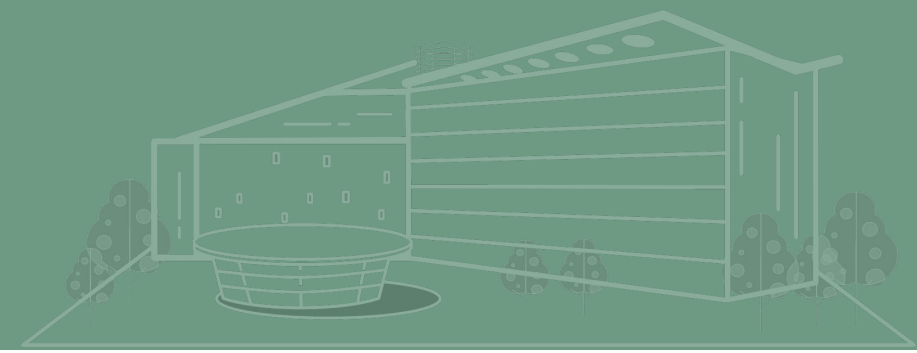
说明：*以上为单轮实测结果，后续可通过多轮测试验证结果稳定性。*

04

综合对比与总结展望

框架对比、适用边界与关键结论





華中師範大學

LangChain 与 Google ADK：实现方式对比

相同的并行化模式，可以采用不同的软件抽象与编排组件

比较维度	LangChain / LCEL	Google ADK
并行对象	多个Runnable / Chain	多个专职LlmAgent
核心组件	RunnableParallel	ParallelAgent
后续汇总	Synthesis Prompt + LLM	Merger Agent
控制方式	Runnable链式组合（|）	SequentialAgent组织阶段
教材示例	主题摘要、问题、关键词	三领域并行研究与综述

共同原则：识别独立任务 → 并发执行 → 必要时统一汇总

并行化的价值与适用边界

并行可以缩短等待，却并非任务越多、分支越多就一定越快

主要价值	适用场景	设计挑战
降低多路 I/O 等待	独立的LLM/API调用	增加分支可能增加 调用次数与费用
提高复杂任务响应速度	多数据源并发检索	调试、日志和监控更复杂
汇总多源与多角度结果	多版本内容独立生成	慢分支、超时及结果冲突

关键判断：子任务之间不存在必须等待的依赖，且并发收益大于额外开销。

关键结论与展望

Parallelization是任务依赖驱动的工作流优化模式，而非简单增加Agent数量

01

独立性优先

识别任务依赖
决定哪些步骤能够并发

02

并发与串行结合

分支同时执行
所需结果就绪后再汇总

03

框架各有侧重

LangChain**管理**Runnable
ADK**调度专业子**Agent

04

评估真实收益

结合延迟、费用、可靠性
与可观测性进行权衡

未来优化方向：依赖分析、动态任务调度、错误恢复与成本感知编排



Thank you

谢谢聆听

第三小组：并行化